Proxied Authentication in Flowsource using OAuth2 Proxy with Microsoft Azure AD provider

Table of Contents

Prerequisites ***2***

Environment Configuration Steps ***2***

Installing Nginx Ingress Controller **2**

Configuring Nginx Ingress Controller **2**

Creating and configuring a Load Balancer (If creating manually, as Ingress is created using NodePort) 2

Configuring Azure OAuth App **3**

Deploying OAuth2 Proxy **3**

Create the OAuth2 Proxy namespace 3

Create the OAuth2 Proxy secrets 3

Create the OAuth2 Proxy ConfigMap 3

Deploy the OAuth2 Proxy application 3

Create the OAuth2 Proxy Service 3

Configuring Ingress Resources **3**

Create the Ingress for the OAuth2 Proxy 3

Create the Ingress for your application 3

Flowsource Application Source Code updates ***4***

app-config.yaml **4**

packages/app/src/App.tsx update **4**

packages/backend/src/plugins/auth.ts update **5**

k8s YAML configurations ***7***

oauth-proxy-config.yaml **7**

oauth-proxy-ingress.yaml **8**

flowsource-ingress.yaml **9**

oauth-proxy-svc.yaml **10**

oauth-proxy-deployment.yaml **11**

Conclusion ***13***

This document provides a guide on how to set up Single Sign-On (SSO) offloading using OAuth2 Proxy with Microsoft Azure Active Directory (AD) in a Kubernetes environment.
(*Note that it is also possible to use other providers, such as GitHub, or any OIDC-compliant identity provider, to achieve similar SSO functionality based on your specific requirements and preferences. If you require configurations for other identity providers, such as GitHub, please refer to the YAML files included in the installer at `Flowsource_Package_VERSION/proxied-authentication/yamls/` and follow similar steps.*)

Prerequisites

- A Kubernetes cluster
- Ingress Nginx Controller
- Microsoft Azure AD
- OAuth App in Microsoft Azure
- kubectl CLI
- Helm CLI (Optional)

Environment Configuration Steps

The setup involves deploying the OAuth2 Proxy in your Kubernetes cluster and configuring it to use Azure AD for authentication. The Nginx Ingress Controller is then configured to use the OAuth2 Proxy for authentication.

Installing Nginx Ingress Controller

1. **Install the Nginx Ingress Controller (Using Helm)**
   Run the following command to install the Nginx Ingress Controller using Helm:

```
helm upgrade --install ingress-nginx ingress-nginx \
  --repo https://kubernetes.github.io/ingress-nginx \
  --namespace ingress-nginx --create-namespace \
  --set controller.service.type=NodePort \
  --set controller.config.allow-snippet-annotations=true
```

Configuring Nginx Ingress Controller

This step is not required if the Ingress Controller is created with LoadBalander service type.

Creating and configuring a Load Balancer (If creating manually, as Ingress is created using NodePort)

1. **Create a Load Balancer**

Create a Load Balancer manually to point to the Ingress Nginx controller. The Load Balancer should be in the same VPC/Subnet as the Kubernetes cluster.

2. **Configure Listeners**

Configure the listeners of the Load Balancer to listen on the source port (HTTPS) and forward the traffic to the target port (HTTP) of the NodePort service exposed by the Nginx Ingress Controller.

3. **Configure SSL/TLS Certificate**

Configure the SSL/TLS certificate for the Load Balancer. This certificate can be obtained from a Certificate Manager.

4. **Configure Health Check**

Configure the health check of the Load Balancer to check the TCP connection on the target port of the NodePort service.

5. **Configure Target Instances**

Configure the target instances of the Load Balancer to be the instances in the Kubernetes cluster node group(s).

6. **Configure Security Groups**

Configure the security group of the Load Balancer to allow inbound traffic on the source port from the IPs or security groups that need to access the application or ingress.
Also, update the security group of the cluster/node group to allow the newly created Load Balancer on the ports range of the NodePort service.

Configuring Azure OAuth App

Register a new Azure OAuth app with the below details. (

Homepage URL: INGRESS_BASE_URL

Redirect URI: INGRESS_BASE_URL/oauth2/callback

(Note: INGRESS_BASE_URL should be updated with the actual value.)

After the app is created, you should be able to find the Application (client) ID , Application Client Secret and Directory (tenant) ID. You'll need these values for your OAuth2 Proxy configuration.

Deploying OAuth2 Proxy

Create the OAuth2 Proxy namespace

Run the following command to create a new namespace for the OAuth2 Proxy:

```
kubectl create namespace oauth2-proxy
```

(For simplicity, the namespace oauth2-proxy is used in the yamls)

Create the OAuth2 Proxy secrets

Replace CLIENT_ID, CLIENT_SECRET, and COOKIE_SECRET with your Azure AD application's client ID, client secret, and a random string for the cookie secret respectively.

Then run the following command:

```
kubectl create secret generic oauth2-proxy-secrets \
  --namespace=oauth2-proxy \
  --from-literal=OAUTH2_PROXY_CLIENT_ID=CLIENT_ID \
  --from-literal=OAUTH2_PROXY_CLIENT_SECRET=CLIENT_SECRET \
  --from-literal=OAUTH2_PROXY_COOKIE_SECRET=COOKIE_SECRET
```

Create the OAuth2 Proxy ConfigMap

Replace TENANT_ID and INGRESS_BASE_URL with your Azure AD tenant ID and the base URL of your ingress respectively. Make sure the redirect_url mentioned in the configmap matches the one in the OAuth app.

Then apply the oauth-proxy-config.yaml file:

```
kubectl apply -f oauth-proxy-config.yaml
```

Deploy the OAuth2 Proxy application

Apply the oauth-proxy-deployment.yaml file:

```
kubectl apply -f oauth-proxy-deployment.yaml
```

Create the OAuth2 Proxy Service

Apply the oauth-proxy-svc.yaml file:

```
kubectl apply -f oauth-proxy-svc.yaml
```

Configuring Ingress Resources

Create the Ingress for the OAuth2 Proxy

Replace INGRESS_DNS with the DNS name of your ingress.

Then apply the oauth-proxy-ingress.yaml file:

```
kubectl apply -f oauth-proxy-ingress.yaml
```

Create the Ingress for your application

Replace INGRESS_BASE_URL and INGRESS_DNS with the base URL and DNS name of your ingress respectively.

Then apply the flowsource-ingress.yaml file:

```
kubectl apply -f flowsource-ingress.yaml
```

Flowsource Application Source Code updates

These updates are already included in the Flowsource package. The files App.tsx, auth.ts, and app-config.yaml are provided for reference purposes only.

Ensure the environment variable SIGN_IN_AUTH_PROVIDER is set to "oauth2Proxy" to enable the frontend to use the proxied sign-in page. This variable can be configured in values.yaml, which subsequently generates the ConfigMap.

app-config.yaml

```yaml
app:
  signInAuthProvider: ${SIGN_IN_AUTH_PROVIDER}
auth:
  environment: development
  providers:
    oauth2Proxy:
      development:
        signIn:
          resolvers:
            - resolver: emailMatchingUserEntityProfileEmail
            - resolver: emailLocalPartMatchingUserEntityName
            - resolver: forwardedUserMatchingUserEntityName
```

packages/app/src/App.tsx update

```typescript
import { ProxiedSignInPage } from '@backstage/core-components';

const app = createApp({
  apis,
  components: {
    SignInPage: props => {
      return (
        <ProxiedSignInPage {...props} provider="oauth2Proxy" />
      );
    },
  },

  // …

});
```

packages/backend/src/plugins/auth.ts update

```
import {
  createRouter,
  providers,
  defaultAuthProviderFactories,
} from '@backstage/plugin-auth-backend';
import { Router } from 'express';
import { PluginEnvironment } from '../types';
import { getUpdatedUserRefs } from './helper/auth-helper';
import { initDatabase } from './database/initDatabase.service';
import { RoleMappingDatabaseService } from
'./database/roleMappingDatabase.service';
import { Knex } from 'knex';
import { DEFAULT_NAMESPACE, stringifyEntityRef, } from '@backstage/catalog-model';
// Added for Azure AD Auth
import { decodeJwt } from 'jose';

const HEADER_KEY_PROVIDER = 'provider';

export default async function createPlugin(
  env: PluginEnvironment,
): Promise<Router> {

  // Initialize/create the database
  await initDatabase({
    logger: env.logger,
    database: env.database,
  });

  // Get database client
  const db: Knex = await env.database.getClient();
  const roleMappingDatabaseService = new RoleMappingDatabaseService(db);

  // Create the router
  return await createRouter({
    logger: env.logger,
    config: env.config,
    database: env.database,
    discovery: env.discovery,
    tokenManager: env.tokenManager,
    providerFactories: {
      ...defaultAuthProviderFactories,

      oauth2Proxy: providers.oauth2Proxy.create({
        signIn: {
          resolver: async ({ result }, ctx) => {
            env.logger.debug('In oauth2Proxy resolver - result - ', result);

            const authProvider = result.getHeader(HEADER_KEY_PROVIDER);
            if (!authProvider) {
```

```
              env.logger.error('Missing auth provider in the request header');
              throw new Error('Missing auth provider in the request header');
            }

            let teamsOrRoles: any = [];
            let userEntityName: any;
            let usernameEntityRef: any;

            if (authProvider === 'microsoft') {
              const idTokenEncoded: any = result.getHeader('Auth');
              const idToken: any = decodeJwt(idTokenEncoded);
              if (!idToken.email) {
                env.logger.error('Login failed, user profile does not contain an
email');
                throw new Error('Login failed, user profile does not contain an
email');
              }
              userEntityName = idToken.email;

              if(idToken.roles) {
                teamsOrRoles = idToken.roles;
              } else {
                console.log("No role found");
              }
            }

            usernameEntityRef = stringifyEntityRef({
              kind: 'User',
              name: userEntityName,
              namespace: DEFAULT_NAMESPACE,
            });

            const userRefs = await getUpdatedUserRefs(teamsOrRoles, authProvider,
roleMappingDatabaseService); // Fetch the updated user references

            return ctx.issueToken({
              claims: {
                sub: usernameEntityRef, // The user's own identity
                ent: userRefs, // A list of identities that the user claims
ownership through
              },
            });
          },
        },
      })

    },
  });

}
```

k8s YAML configurations

oauth-proxy-config.yaml

```yaml
apiVersion: v1
data:
  oauth2-proxy.cfg: |-
    provider="azure"
    oidc_issuer_url="https://login.microsoftonline.com/TENANT_ID/v2.0"
    cookie_secure=true
    scope="openid email"
    http_address="0.0.0.0:4180"
    upstreams=["INGRESS_BASE_URL"]
    redirect_url = "INGRESS_BASE_URL/oauth2/callback"
    email_domains=["*"]
    whitelist_domains=["INGRESS_DNS:*"]
    set_xauthrequest=false
    skip_auth_strip_headers=false
    set_authorization_header=true
    pass_authorization_header=true
    pass_basic_auth=true
    pass_user_headers=true
    pass_host_header=true
kind: ConfigMap
metadata:
  name: oauth2-proxy-config
  namespace: oauth2-proxy
```

oauth-proxy-ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: oauth2-proxy
  namespace: oauth2-proxy
  annotations:
    nginx.ingress.kubernetes.io/proxy-buffer-size: "512k"
spec:
  ingressClassName: nginx
  rules:
  - http:
      paths:
      - backend:
          service:
            name: oauth-proxy
            port:
              number: 4180
        path: /oauth2
        pathType: Prefix
    host: INGRESS_DNS
```

flowsource-ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  annotations:
    nginx.ingress.kubernetes.io/auth-signin:
INGRESS_BASE_URL/oauth2/start?rd=$escaped_request_uri
    nginx.ingress.kubernetes.io/auth-url: http://oauth-proxy.oauth2-
proxy.svc.cluster.local:4180/oauth2/auth
    nginx.ingress.kubernetes.io/auth-response-headers: x-auth-request-user, x-auth-
request-email
    nginx.ingress.kubernetes.io/configuration-snippet: |
      auth_request_set $auth $upstream_http_authorization;
      proxy_set_header "Auth" $auth;
      proxy_set_header "provider" "microsoft";
    nginx.ingress.kubernetes.io/proxy-buffer-size: "512k"
    nginx.ingress.kubernetes.io/server-snippet: |
      large_client_header_buffers 8 32k;
  name: nginx-ingress
  labels:
    app: flowsource
  name: flowsource-ingress
spec:
  ingressClassName: nginx
  rules:
  - host: INGRESS_DNS
    http:
      paths:
      - backend:
          service:
            name: flowsource
            port:
              number: 7007
        path: /
        pathType: Prefix
```

oauth-proxy-svc.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  labels:
    app: oauth-proxy
  name: oauth-proxy
  namespace: oauth2-proxy
spec:
  type: ClusterIP
  selector:
    app: oauth-proxy
  ports:
  - name: http-oauthproxy
    port: 4180
```

oauth-proxy-deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: oauth-proxy
  name: oauth-proxy
  namespace: oauth2-proxy
spec:
  replicas: 1
  selector:
    matchLabels:
      app: "oauth-proxy"
  template:
    metadata:
      labels:
        app: oauth-proxy
    spec:
      volumes:
        - name: oauth2-proxy-config
          configMap:
            name: oauth2-proxy-config
        # - name: oauth2-secrets-store01-inline
        #   csi:
        #     driver: secrets-store.csi.k8s.io
        #     readOnly: true
        #     volumeAttributes:
        #       secretProviderClass: "oauth-proxy-secrets-provider"
      containers:
      - name: oauth-proxy
        image: "quay.io/oauth2-proxy/oauth2-proxy:latest"
        ports:
        - containerPort: 4180
        envFrom:
          - secretRef:
              name: oauth2-proxy-secrets
        volumeMounts:
        - name: oauth2-proxy-config
          mountPath: /etc/oauth2-proxy.cfg
          subPath: oauth2-proxy.cfg
        # - name: oauth2-secrets-store01-inline
        #   mountPath: /mnt/secrets-store
        #   readOnly: true
        args:
          - --config=/etc/oauth2-proxy.cfg
```

Troubleshooting

1) **Error Message**: "Needs permission to access resources in your organization that only an admin can grant. Please ask an admin to grant permission to this app before you can use it."

   Cause: This issue is related to the Azure AD configurations.

   Recommended Actions:
   1) Contact Azure AD team
      We recommend checking with your "Azure AD" team to resolve this issue.
   2) Enterprise Application Settings
      In the Azure portal, in Enterprise Application's service, look for the User Settings section. Check the configurations related to these 2 points, and update them as necessary:
      i) Users can consent to apps accessing company data on their behalf.
      ii) Users can request admin consent to apps they are unable to consent to.

   Additional Resources:
   - Check this post in the official Microsoft site.
   - Refer to this post from Stackoverflow for more details.

2) **Error Message**: "annotation group ServerSnippet contains risky annotation based on ingress configuration" (*This is regarding the Nginx Controller*)

   Potential solution: "controller.config.annotations-risk-level: Critical" can be set in the Nginx Ingress Controller Helm installation

   Additional Resources:
   - Check this post in the GitHub Issues.

Conclusion

After following these steps, your application should be protected by the OAuth2 Proxy, which uses Azure AD for authentication. Users will be redirected to Microsoft login page for sign-in and then redirected back to your application.